

11: Rust Senior Interview Guide, Part 1: Collections, Idioms, ADTs, Smart Pointers

In a senior Rust interview, working code is expected. What distinguishes a strong answer is the choice of types: a collection whose costs match the access pattern, an enum that makes invalid states impossible to construct, and ownership that is decided when the data structure is designed rather than patched later with clones.

This is the first of three Rust guides. It covers the standard collections, the naming and API conventions from the Rust API Guidelines, algebraic data types with pattern matching, and smart pointers. The examples are short enough to write from memory in an interview.

This chapter covers

- How each standard collection is stored, what its operations cost, and when to use it
- Naming, documentation, derives, constructors, errors, and builders in idiomatic Rust
- Modeling states and errors with enum and match
- Box, Rc, Arc, RefCell, Mutex, RwLock, and Weak, and how to choose among them
- What the Send and Sync traits mean

Contents

1. The standard collections	3
1.1 Vec<T>: a growable array	3
1.2 VecDeque<T>: a double-ended queue	4
1.3 LinkedList<T>: a doubly linked list	5
1.4 HashMap<K, V>: a hash table	6
1.5 BTreeMap<K, V>: an ordered map	7
1.6 HashSet<T>: a hash set	7
1.7 BTreeSet<T>: an ordered set	8
1.8 BinaryHeap<T>: a priority queue	8
1.9 Summary table	10
2. Idiomatic Rust APIs	11
2.1 Naming	11
2.2 Documentation	11
2.3 Deriving common traits	12
2.4 Constructors	13
2.5 Errors	13
2.6 The builder pattern	13

3. Algebraic data types and pattern matching	15
3.1 States as an enum.....	15
3.2 match instead of if-else chains.....	15
3.3 if let and while let	16
3.4 Matching variants that carry data	16
3.5 Match guards.....	17
4. Smart pointers and interior mutability	18
4.1 Choosing a pointer type.....	18
4.2 Box<T>	18
4.3 Rc<T> and RefCell<T>.....	19
4.4 Arc<T> and Mutex<T>	20
4.5 RwLock<T>	20
4.6 Weak<T>	21
4.7 Send and Sync	22
5. Summary	24

1. The standard collections

For each collection, this section describes its memory layout, the cost of its main operations, when to choose it, an example, and common mistakes. The layout explains the costs, so learn them together.

1.1 Vec<T>: a growable array

A Vec stores its elements in one contiguous heap allocation. The Vec value itself holds a pointer, a length, and a capacity:

```
ptr → [ T | T | T | ... unused capacity ]
len = used elements
capacity = allocated slots
```

Operation	Cost
push, pop	O(1) amortized
insert, remove at index i	O(n), because later elements shift
Index access	O(1)
Iteration	O(n), with good cache behavior
Memory	Three machine words, plus capacity * size_of::<T>() on the heap

When to use it. Vec is the default choice for any sequence. It is also usually the right choice when you are considering a linked list, because contiguous memory makes iteration and most modifications faster in practice.

```
fn main() {
    let mut gpus = Vec::with_capacity(4);
    gpus.push("A100");
    gpus.push("H100");
    gpus.push("L40S");

    assert_eq!(gpus[0], "A100");
    assert_eq!(gpus.get(10), None); // safe lookup

    if let Some(last) = gpus.pop() {
        println!("popped {last}");
    }

    gpus.insert(1, "B200");
    gpus.remove(0);

    for (i, gpu) in gpus.iter().enumerate() {
        println!("gpus[{i}] = {gpu}");
    }
}
```

Common mistakes:

- **Unnecessary reallocation.** When `len` reaches capacity, `push` allocates a larger buffer and moves every element. Use `Vec::with_capacity` when you know the final size.
- **Removing elements by index inside a loop.** Each removal shifts the remaining indexes. Use `retain` to remove elements that match a condition, or `drain` to remove a range.
- **Panicking on an invalid index.** `v[i]` panics when `i` is out of bounds; `v.get(i)` returns `Option<&T>`.
- **Expecting bit packing.** A `Vec<bool>` uses one byte per element. Use a crate such as `bitvec` if you need one bit per element.

1.2 VecDeque<T>: a double-ended queue

A `VecDeque` stores elements in a ring buffer: a single allocation in which the elements can wrap around from the end back to the start.

Operation	Cost
<code>push_front</code> , <code>pop_front</code> , <code>push_back</code> , <code>pop_back</code>	$O(1)$ amortized
Index access	$O(1)$
<code>make_contiguous()</code>	$O(n)$; rearranges the elements into one slice

When to use it. Use `VecDeque` for FIFO queues, work queues in a worker pool, BFS, sliding windows, and retry queues: any case where you add at one end and remove at the other.

```
use std::collections::VecDeque;

fn main() {
    let mut queue = VecDeque::new();
    queue.push_back("job-a");
    queue.push_back("job-b");
    queue.push_front("urgent-job");

    assert_eq!(queue.pop_front(), Some("urgent-job"));
    assert_eq!(queue.pop_front(), Some("job-a"));
    queue.push_back("job-c");

    // Rotate to simulate round-robin scheduling.
    if let Some(job) = queue.pop_front() {
        queue.push_back(job);
    }

    println!("queue: {queue:?}");
}
```

Common mistakes:

- **Expecting a single slice.** Because the elements can wrap around, `VecDeque` does not dereference to `&[T]`. Call `as_slices()` to get the two parts, or `make_contiguous()` to get one slice, which you can then sort.
- **Expecting memory to be released.** The buffer keeps its capacity after elements are removed. Call `shrink_to_fit` if a queue that was once large should release memory.

1.3 LinkedList<T>: a doubly linked list

Each element is a separate heap allocation with pointers to the previous and next nodes.

Operation	Cost
Push and pop at either end	$O(1)$
Access by position	$O(n)$
append another list	$O(1)$
<code>split_off(at)</code>	$O(\min(at, len - at))$, to walk to the split point

When to use it. Rarely. `VecDeque` supports the same operations at both ends and is faster, because its elements are adjacent in memory and a linked list requires a pointer dereference, often a cache miss, for each element. A linked list is justified only when you need $O(1)$ splicing of whole lists or nodes whose addresses never change. In interviews, linked lists appear mainly to test your understanding of ownership; see `13-rust-guide-part3-gotchas-linkedlist-interview.md`.

```
use std::collections::LinkedList;

fn main() {
    let mut list = LinkedList::new();
    list.push_back(2);
    list.push_back(3);
    list.push_front(1);

    let mut rest = list.split_off(1); // list=[1], rest=[2,3]
    list.append(&mut rest);

    while let Some(value) = list.pop_front() {
        print!("{value} ");
    }
    println();
}
```

Common mistakes:

- **Using it for performance.** Iteration is slower than for `Vec` or `VecDeque`.
- **Indexing.** `LinkedList` does not implement indexing, so `list[0]` does not compile.

1.4 HashMap<K, V>: a hash table

The standard HashMap is a port of Google's SwissTable design (the hashbrown crate), an open-addressing table that stores control bytes alongside the entries. By default it hashes keys with SipHash-1-3 using a random key, which prevents attackers from choosing keys that all collide (a HashDoS attack).

Operation	Cost
get, insert, remove	O(1) expected
Iteration	O(capacity), in an unspecified order that varies between runs

When to use it. Use HashMap for lookups by key, counters, caches, and indexes.

```
use std::collections::HashMap;

fn main() {
    let mut quotas: HashMap<String, u32> = HashMap::new();

    quotas.insert("team-a".into(), 8);
    *quotas.entry("team-b".into()).or_insert(4) += 1;

    match quotas.get("team-a") {
        Some(q) => println!("team-a quota {q}"),
        None => println!("team-a has no quota"),
    }

    let old = quotas.remove("team-b");
    println!("removed: {old:?}");
}
```

The entry API performs an insert-or-update with a single lookup.

`*quotas.entry(key).or_insert(4) += 1` inserts 4 if the key is absent and then increments the value.

Common mistakes:

- **Depending on iteration order.** The order can change between runs. Sort the keys, or use BTreeMap, when output must be deterministic, for example in tests.
- **Changing the hasher without measurement.** A faster non-cryptographic hasher such as FxHash can speed up maps with integer keys, but it removes the HashDoS protection. Change it only when profiling shows hashing is a bottleneck and the keys are not controlled by users.
- **Borrow conflicts.** Code such as "get, and if missing, insert" can hold a borrow from get while calling insert. The entry API avoids the conflict.

1.5 BTreeMap<K, V>: an ordered map

A BTreeMap is a B-tree: each node holds several sorted keys, which keeps the tree shallow and makes good use of the CPU cache.

Operation	Cost
get, insert, remove	$O(\log n)$
Iteration	$O(n)$, in key order
range, range_mut	$O(\log n)$ to find the start, then $O(1)$ per element

When to use it. Use BTreeMap when you need ordered iteration, range queries, deterministic output, or the first or last key. For small maps, it can also be faster than HashMap because it does not hash.

```
use std::collections::BTreeMap;

fn main() {
    let mut nodes = BTreeMap::new();
    nodes.insert("node-3", 3);
    nodes.insert("node-1", 1);
    nodes.insert("node-2", 2);

    for (name, id) in &nodes {
        println!("{name}: {id}");
    }

    // Keys are sorted, so range scans are natural.
    for (name, id) in nodes.range("node-2"..) {
        println!("range >= node-2: {name}: {id}");
    }
}
```

Common mistakes:

- **Using key types without Ord.** Keys must implement Ord. f64 does not; wrap it in a type that defines a total order if you need floating-point keys.

1.6 HashSet<T>: a hash set

A HashSet<T> is a HashMap<T, ()>.

Operation	Cost
insert, remove, contains	$O(1)$ expected
union, intersection, difference, is_subset	Proportional to the sizes of the sets

When to use it. Use HashSet for deduplication, membership tests, and visited sets in graph searches.

```

use std::collections::HashSet;

fn main() {
    let mut allowed: HashSet<&str> = HashSet::new();
    allowed.insert("a100");
    allowed.insert("h100");

    println!("contains h100? {}", allowed.contains("h100"));
    allowed.remove("a100");

    for product in allowed {
        println!("allowed: {product}");
    }
}

```

Common mistakes:

- **Depending on iteration order**, as with HashMap. Use BTreeSet for sorted order.
- **Choosing the element type without considering lifetimes.** A HashSet<&str> borrows from strings that must outlive the set; a HashSet<String> owns its elements. Both can be queried with a &str.

1.7 BTreeSet<T>: an ordered set

A BTreeSet<T> is a BTreeMap<T, ()>. Its operations cost $O(\log n)$, iteration is sorted, and it supports range queries.

When to use it. Use BTreeSet for sorted unique values and range scans.

```

use std::collections::BTreeSet;

fn main() {
    let mut zones = BTreeSet::new();
    zones.insert("us-east-1");
    zones.insert("us-west-2");
    zones.insert("eu-central-1");

    assert!(zones.contains("us-west-2"));

    for zone in zones.range("eu".."us") {
        println!("{}", zone);
    }
}

```

1.8 BinaryHeap<T>: a priority queue

A BinaryHeap stores a binary heap in a Vec: the children of the element at index i are at indexes $2i + 1$ and $2i + 2$.

Operation	Cost
push	$O(\log n)$ worst case, $O(1)$ on average
pop	$O(\log n)$
peek	$O(1)$

BinaryHeap is a **max-heap**: pop returns the largest element. For a min-heap, wrap elements in `std::cmp::Reverse`.

When to use it. Use BinaryHeap for top-k problems, scheduling by priority or deadline, and Dijkstra's algorithm.

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

fn main() {
    let mut max_heap = BinaryHeap::new();
    max_heap.push(3);
    max_heap.push(1);
    max_heap.push(4);
    assert_eq!(max_heap.pop(), Some(4));

    let mut min_heap = BinaryHeap::new();
    min_heap.push(Reverse(3));
    min_heap.push(Reverse(1));
    min_heap.push(Reverse(4));
    assert_eq!(min_heap.pop(), Some(Reverse(1)));
}
```

Common mistakes:

- **Expecting to update or remove arbitrary elements.** BinaryHeap has no efficient operation for either. A common workaround is to push a new entry and ignore stale entries when they are popped.
- **Forgetting that it is a max-heap.** Use Reverse for a min-heap. Reverse is a zero-cost wrapper: it changes only the comparison and adds no allocation or size.

1.9 Summary table

Need	Collection	Main cost	Note
Any sequence	<code>Vec<T></code>	$O(1)$ amortized push and pop	Contiguous; cache-friendly
Queue, deque, BFS	<code>VecDeque<T></code>	$O(1)$ at both ends	Ring buffer
Ordered key-value	<code>BTreeMap<K, V></code>	$O(\log n)$	Range queries
Unordered key-value	<code>HashMap<K, V></code>	$O(1)$ expected	Unspecified order
Membership, deduplication	<code>HashSet<T></code>	$O(1)$ expected	Unspecified order
Sorted membership	<code>BTreeSet<T></code>	$O(\log n)$	Range queries
Priority queue	<code>BinaryHeap<T></code>	$O(\log n)$ pop	Max-heap
Splicing lists	<code>LinkedList<T></code>	$O(1)$ at ends, $O(n)$ access	Rarely the best choice

2. Idiomatic Rust APIs

The conventions in this section come from the Rust API Guidelines. Following them makes your code look familiar to other Rust developers, and interviewers notice when it does not.

2.1 Naming

Item	Convention	Example
Functions, methods, variables, modules	snake_case	gpu_count
Types, traits, enum variants	CamelCase	GpuWorkload
Constants and statics	SCREAMING_SNAKE_CASE	DEFAULT_GPU_COUNT
Boolean queries	is_, has_, or a verb	is_empty(), contains()
Cheap borrowed conversion	as_	as_str()
Expensive or owned conversion	to_	to_string()
Conversion that consumes self	into_	into_inner()
Getters	The field name, without get_	image()

The receiver shows what a method does with the value: `&self` reads it, `&mut self` modifies it, and `self` consumes it.

```
const DEFAULT_GPU_COUNT: u32 = 1;

pub struct GpuWorkload {
    image: String,
    gpu_count: u32,
}

impl GpuWorkload {
    pub fn new(image: impl Into<String>) -> Self { ... }

    pub fn image(&self) -> &str { &self.image }
    pub fn gpu_count(&self) -> u32 { self.gpu_count }
    pub fn with_gpu_count(mut self, count: u32) -> Self { ... }
}
```

2.2 Documentation

- `///` documents the item that follows it.
- `//!` documents the enclosing module or crate, and goes at the top of the file.
- Include an example for any public function whose use is not obvious. Examples in doc comments are compiled and run by `cargo test`, so they stay correct.

```

/// Returns the number of complete GPU jobs of size `requested` that fit in
/// `available`.
///
/// ```
/// use mylib::jobs_that_fit;
/// assert_eq!(jobs_that_fit(4, 10), 2);
/// ```
pub fn jobs_that_fit(requested: u32, available: u32) -> u32 {
    available / requested
}

```

The example function divides by requested, so it panics when requested is zero. A production version would document that in a # Panics section or return an Option.

2.3 Deriving common traits

```

#[derive(Debug, Clone, PartialEq, Eq, Hash, Default)]
pub struct SchedulerConfig {
    pub max_gpus: u32,
    pub region: String,
}

impl SchedulerConfig {
    pub fn with_capacity(&mut self, n: usize) -> &mut Self { ... }
}

```

Derive a trait when its meaning is correct for the type:

Trait	Derive it when
Debug	Almost always; it is needed for <code>{:?}</code> and for test failure messages
Clone	Callers need copies, and copying is not surprisingly expensive
PartialEq, Eq	Field-by-field equality is the right definition of equality
Hash	The type is used as a HashMap key or HashSet element; it must agree with Eq
Default	An all-default value is a valid configuration

The Rust API Guidelines recommend that public types implement the common traits eagerly, because users of your crate cannot add them later.

2.4 Constructors

Constructor	Use
<code>new()</code>	The primary constructor
<code>with_capacity(n)</code>	Collections, or types that preallocate
<code>from_...()</code> , or <code>impl From<T></code>	Conversions from another type
Default	Types with a sensible default value
A builder	Types with many optional fields

2.5 Errors

- **Libraries** define an error enum for their failure cases, commonly with the `thiserror` crate, so callers can match on the cause.
- **Applications** often use `anyhow::Result` and add context as errors propagate.
- **Public functions that can fail** return `Result<T, E>` with a specific error type.
- **Avoid `unwrap()` in production code** unless a local invariant guarantees success. In that case, prefer `expect("...")` with a message that states the invariant.

12-rust-guide-part2-errors-concurrency-sdk.md covers error handling in detail.

2.6 The builder pattern

A builder lets callers set only the fields they need and validates the result once, in `build`:

```

#[derive(Debug, Clone)]
pub struct WorkloadSpec {
    image: String,
    gpu_count: u32,
    runtime_class: Option<String>,
}

#[derive(Default)]
pub struct WorkloadSpecBuilder {
    image: Option<String>,
    gpu_count: u32,
    runtime_class: Option<String>,
}

impl WorkloadSpecBuilder {
    pub fn image(mut self, image: impl Into<String>) -> Self {
        self.image = Some(image.into());
        self
    }

    pub fn gpu_count(mut self, count: u32) -> Self {
        self.gpu_count = count;
        self
    }

    pub fn runtime_class(mut self, name: impl Into<String>) -> Self {
        self.runtime_class = Some(name.into());
        self
    }

    pub fn build(self) -> Result<WorkloadSpec, String> {
        Ok(WorkloadSpec {
            image: self.image.ok_or("image is required")?,
            gpu_count: self.gpu_count,
            runtime_class: self.runtime_class,
        })
    }
}

fn main() -> Result<(), String> {
    let spec = WorkloadSpecBuilder::default()
        .image("nvcr.io/nvidia/tritonserver:24.05-py3")
        .gpu_count(1)
        .runtime_class("nvidia")
        .build()?;
    println!("{spec:?}");
    Ok(())
}

```

The setters take `self` by value and return it, so calls can be chained. `build` returns an error when a required field is missing, so an invalid `WorkloadSpec` cannot be created.

3. Algebraic data types and pattern matching

A struct is a *product type*: a value contains all of its fields. An enum is a *sum type*: a value is exactly one of its variants, and each variant can carry its own data. Combining the two lets you model a domain so that invalid combinations cannot be represented. For example, a failure reason exists only in the Failed state, rather than as an optional field that might be set in any state.

3.1 States as an enum

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum WorkloadState {
    Pending,
    Provisioning,
    Running,
    Terminating,
    Terminated,
    Failed { reason: String },
}

impl WorkloadState {
    pub fn summary(&self) -> String {
        match self {
            WorkloadState::Pending => "waiting for resources".into(),
            WorkloadState::Provisioning => "allocating GPU".into(),
            WorkloadState::Running => "running".into(),
            WorkloadState::Terminating => "cleaning up".into(),
            WorkloadState::Terminated => "stopped".into(),
            WorkloadState::Failed { reason } => format!("failed: {reason}"),
        }
    }

    pub fn can_transition_to(&self, next: &Self) -> bool {
        matches!(
            (self, next),
            (WorkloadState::Pending, WorkloadState::Provisioning)
            | (WorkloadState::Provisioning, WorkloadState::Running)
            | (WorkloadState::Running, WorkloadState::Terminating)
            | (WorkloadState::Terminating, WorkloadState::Terminated)
            | (WorkloadState::Failed { .. }, WorkloadState::Pending)
        )
    }
}
```

match must be exhaustive. If you add a variant to WorkloadState, every match that does not handle it fails to compile, so the compiler lists every place that needs to change.

can_transition_to matches on a tuple of the current and next states, which expresses the allowed transitions as a table.

3.2 match instead of if-else chains

match can test ranges and literal patterns directly:

```
fn classify(status_code: u16) -> &'static str {
    match status_code {
        200..=299 => "success",
        400..=499 => "client error",
        500..=599 => "server error",
        _ => "unknown",
    }
}

fn main() {
    println!("{}", classify(429));
}
```

3.3 if let and while let

Use `if let` when you care about one pattern and the other case is simple. Use `while let` to loop for as long as a pattern matches, such as draining a queue:

```
let mut queue = std::collections::VecDeque::from([1, 2, 3]);

while let Some(job) = queue.pop_front() {
    println!("processing {job}");
}

if let Some(value) = queue.pop_front() {
    println!("unexpected: {value}");
} else {
    println!("queue is empty");
}
```

3.4 Matching variants that carry data

Patterns bind the fields of a variant to local names:

```
#[derive(Debug)]
pub enum ApiError {
    Invalid { field: String, message: String },
    RateLimited { retry_after_secs: u64 },
    NotFound { resource: String },
}

fn message(error: &ApiError) -> String {
    match error {
        ApiError::Invalid { field, message } => format!("{field}: {message}"),
        ApiError::RateLimited { retry_after_secs } => {
            format!("retry after {retry_after_secs}s")
        }
        ApiError::NotFound { resource } => format!("{resource} not found"),
    }
}
```


In a real library, implement `std::fmt::Display` for `ApiError` rather than a separate message function, so the error works with `{}` formatting and with `?` conversions.

3.5 Match guards

A guard adds a condition to a pattern. Arms are tried in order, so put the more specific arm first:

```
fn error_severity(err: &ApiError) -> &'static str {
    match err {
        ApiError::RateLimited { retry_after_secs } if *retry_after_secs < 2 => "mild",
        ApiError::RateLimited { .. } => "severe",
        ApiError::NotFound { .. } => "actionable",
        ApiError::Invalid { .. } => "programming error",
    }
}
```

4. Smart pointers and interior mutability

4.1 Choosing a pointer type

Type	Ownership	Mutation	Can cross threads	Use
Box<T>	One owner	Through the owner	If T: Send	Heap allocation, recursive types, trait objects
Rc<T>	Shared; non-atomic reference count	Shared references only	No	Shared ownership within one thread
Arc<T>	Shared; atomic reference count	Shared references only	If T: Send + Sync	Shared ownership across threads
RefCell<T>	One owner	Through &self, checked at run time	Can move, cannot be shared	Interior mutability within one thread
Mutex<T>	One owner, usually inside an Arc	Through a lock guard	Yes	Shared mutable state across threads
RwLock<T>	One owner, usually inside an Arc	Many readers or one writer	Yes	Read-heavy shared state
Weak<T>	Does not own	Must be upgraded first	Like Rc or Arc	Back references, breaking cycles

The common combinations are `Rc<RefCell<T>>` for shared mutable data in one thread and `Arc<Mutex<T>>` for shared mutable data across threads.

4.2 Box<T>

Box puts a value on the heap and owns it. You need it for:

- **Recursive types.** A type cannot contain itself directly, because its size would be infinite. A Box has a fixed size.
- **Trait objects.** `Box<dyn Error>` holds a value of any type that implements the trait.
- **Large values** that you want to move without copying many bytes.

```
#[derive(Debug)]
enum Expr {
    Lit(i64),
    Add(Box<Expr>, Box<Expr>),
}

fn eval(expr: &Expr) -> i64 {
    match expr {
        Expr::Lit(n) => *n,
        Expr::Add(l, r) => eval(l) + eval(r),
    }
}

fn main() {
    let expr = Expr::Add(Box::new(Expr::Lit(1)), Box::new(Expr::Lit(2)));
    println!("{}", eval(&expr));
}
```

4.3 Rc<T> and RefCell<T>

Rc lets several owners share one value; the value is dropped when the last Rc is dropped. Through an Rc you get only shared references, so you cannot modify the value. RefCell provides mutation through a shared reference by moving the borrow check to run time: borrow() and borrow_mut() track active borrows and panic if the rules are broken. Together, Rc<RefCell<T>> gives shared, mutable data within one thread.

```
use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    let shared = Rc::new(RefCell::new(0i32));
    let a = Rc::clone(&shared);
    let b = Rc::clone(&shared);

    *a.borrow_mut() += 1;
    *b.borrow_mut() += 10;

    println!("value = {}", shared.borrow()); // 11
}
```

Warning: borrow_mut() panics if any other borrow of the same RefCell is active, and borrow() panics if a mutable borrow is active. The most common cause is holding a Ref or RefMut guard in a variable while calling a function that borrows the same cell. Keep guards in the smallest possible scope, or use try_borrow_mut() to get a Result instead of a panic.

4.4 Arc<T> and Mutex<T>

Arc is the thread-safe version of Rc: its reference count is updated with atomic operations. Mutex provides mutation through a lock. Together they share mutable state between threads:

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0u32));
    let mut handles = Vec::new();

    for _ in 0..4 {
        let counter = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            for _ in 0..1_000 {
                let mut guard = counter.lock().unwrap();
                *guard += 1;
            }
        }));
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("counter = {}", *counter.lock().unwrap());
}
```

Guidelines for using a mutex:

- **Hold the guard briefly.** The lock is released when the guard is dropped, at the end of its scope. Copy out what you need and let the guard go.
- **Do not call unknown code while holding the guard.** A callback that tries to take the same lock deadlocks.
- **Do not hold a std::sync::Mutex guard across an .await.** The task can be suspended while holding the lock. Use tokio::sync::Mutex if a lock must be held across an await point.
- **Use try_lock()** when waiting for the lock is not acceptable.
- **Understand poisoning.** If a thread panics while holding the guard, the mutex is marked poisoned, and later calls to lock() return Err(PoisonError). You can recover the data with into_inner() on the error, or propagate the failure.

For a simple counter like this one, AtomicU32 avoids the lock entirely.

4.5 RwLock<T>

An RwLock allows many readers at the same time or one writer. It helps when reads are much more frequent than writes and each read holds the lock for a meaningful amount of

time. For very short critical sections, a Mutex is often faster, because an RwLock has more bookkeeping.

```
use std::sync::{Arc, RwLock};

fn main() {
    let config = Arc::new(RwLock::new(vec!["a100".to_string()]));

    // Reader
    let config = Arc::clone(&config);
    let reader = std::thread::spawn(move || {
        let gpus = config.read().unwrap();
        println!("readers: {gpus:?}");
    });

    // Writer
    let config = Arc::clone(&config);
    let writer = std::thread::spawn(move || {
        config.write().unwrap().push("h100".to_string());
    });

    reader.join().unwrap();
    writer.join().unwrap();
}
```

Warning: This example does not compile as written. The second `let config` shadows the original `Arc`, and the reader's `move` closure takes ownership of it, so the writer's `Arc::clone(&config)` uses a moved value (error E0382). Give each clone its own name, for example `let reader_config = Arc::clone(&config);` and `let writer_config = Arc::clone(&config);`, and use those names inside the closures.

Whether writers can be starved by a continuous stream of readers depends on the operating system's lock implementation. Keep read sections short.

4.6 Weak<T>

If two values own each other through `Rc`, their reference counts never reach zero, and neither is freed. A `Weak` pointer refers to a value without owning it, so it does not keep the value alive. To use the value, call `upgrade()`, which returns `None` if the value has already been dropped.

Use `Rc` for references from parent to child and `Weak` for references from child to parent. The doubly linked list in this repository stores its `prev` pointers as `Weak` for the same reason.

```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Node {
    value: i32,
    parent: RefCell<Weak<Node>>,
}

fn main() {
    let parent = Rc::new(Node {
        value: 1,
        parent: RefCell::new(Weak::new()),
    });
    let child = Rc::new(Node {
        value: 2,
        parent: RefCell::new(Rc::downgrade(&parent)),
    });

    // Child -> parent is weak, so dropping child does not keep parent alive.
    let upgraded = child.parent.borrow().upgrade();
    println!("parent value: {:?}", upgraded.map(|p| p.value));
}

```

The child's reference to its parent is weak, so the child does not keep the parent alive: when every Rc to the parent is dropped, the parent is freed, and the child's `upgrade()` then returns `None`.

4.7 Send and Sync

Two marker traits determine what can cross thread boundaries:

- `Send`: a value of the type can be moved to another thread.
- `Sync`: a shared reference `&T` can be used from several threads at once. Equivalently, `T` is `Sync` if `&T` is `Send`.

The compiler implements both automatically for types whose fields all implement them.

Type	Send	Sync	Reason
<code>Rc<T></code>	No	No	The reference count is not atomic
<code>Arc<T></code>	If <code>T</code> : <code>Send</code> + <code>Sync</code>	If <code>T</code> : <code>Send</code> + <code>Sync</code>	The count is atomic, and the value is shared
<code>RefCell<T></code>	If <code>T</code> : <code>Send</code>	No	The borrow flag is not synchronized
<code>Mutex<T></code>	If <code>T</code> : <code>Send</code>	If <code>T</code> : <code>Send</code>	The lock synchronizes access
Raw pointers	No	No	The compiler cannot know how they are used

A type that wraps raw pointers must implement `Send` or `Sync` manually with `unsafe impl`, and document why that is sound.

To confirm at compile time that a type can be shared across threads, write a function with the bound and call it in a test:

```
fn assert_send_sync<T: Send + Sync>() {}

struct Scheduler {
    client: std::sync::Arc<tokio::sync::Mutex<>>,
}

#[test]
fn scheduler_is_send_sync() {
    assert_send_sync::<Scheduler>();
}
```

If someone later adds an `Rc` field to `Scheduler`, this test stops compiling.

5. Summary

- The memory layout of each collection explains its costs. `Vec` and `VecDeque` are contiguous and usually faster than `LinkedList`.
- Use `HashMap` for expected $O(1)$ lookup and `BTreeMap` for ordering and ranges. The `entry` API performs insert-or-update with one lookup.
- Follow the API Guidelines' naming conventions, derive common traits, and use builders for types with many optional fields.
- Model states and errors with `enum`, and let exhaustive `match` find every place that must handle a new variant.
- Use `Rc<RefCell<T>>` for shared mutable data in one thread, `Arc<Mutex<T>>` across threads, and `Weak` for back references.
- `Send` and `Sync` are derived from a type's fields; `Rc` and `RefCell` prevent a type from being shared across threads.

Source: [11-rust-guide-part1-collections-adt-smart-pointers.md](#) · [PDF](#) · [All study guides](#)